

Most asked Python Interview Questions (0-3 Years) 10-30 LPA

Python Questions

1. How do you implement memoization in Python to optimize recursive functions?

```
from functools import lru_cache

@lru_cache(maxsize=None)
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

print(fibonacci(10)) # Output: 55
```

Explanation:

- Memoization stores the results of expensive function calls and returns the cached result when the same inputs occur again.
- @lru_cache is a built-in Python decorator that handles this caching automatically.
- In recursive algorithms like Fibonacci, it reduces time complexity from exponential to linear.

Tip:

Use @lru_cache from functools for dynamic programming problems or when repeated function calls occur with the same inputs — it improves both performance and readability.

2. What is the difference between Generators and Iterators in Python, and how do you create a generator?

```
def countdown(n):
    while n > 0:
        yield n
        n -= 1

gen = countdown(3)
print(next(gen)) # Output: 3
print(next(gen)) # Output: 2

# Custom iterator class
class Countdown:
```

```
def __init__(self, n):
    self.n = n

def __iter__(self):
    return self

def __next__(self):
    if self.n <= 0:
        raise StopIteration
    self.n -= 1
    return self.n + 1

it = Countdown(3)
for i in it:
    print(i) # Output: 3, 2, 1
```

Explanation:

Generator: A function using yield to return items one at a time. It automatically manages state and raises StopIteration.

Iterator: A class-based implementation with __iter__() and __next__() methods.

Both allow lazy evaluation, but generators are shorter and more Pythonic for simple use cases.

Tip:

Use generators when performance and memory efficiency matter, especially with large datasets or streaming data — they're a common interview expectation for scalable solutions.

Returns **all records from both tables**, matching where possible.

3. How do you use ***args** and ****kwargs** in advanced function wrappers (decorators) to create flexible decorators?

```
import functools
```

```
def logger_decorator(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        # Log function name and arguments
        print(f"Calling {func.__name__} with args: {args} and kwargs: {kwargs}")
        result = func(*args, **kwargs)
        print(f"{func.__name__} returned {result}")
        return result
    return wrapper
```

```
@logger_decorator
def compute_sum(a, b, c=0):
    return a + b + c
```

```
# Usage
compute_sum(1, 2, c=3)
```

Explanation:

- The decorator `logger_decorator` demonstrates the use of `*args` and `**kwargs` to pass an arbitrary number of positional and keyword arguments to the decorated function.
- `*args` collects extra positional arguments, while `**kwargs` collects extra keyword arguments.
- Using `functools.wraps(func)` preserves the original function's metadata (like its name and docstring).

Tip:

When creating decorators, always consider how to transparently pass through all arguments and preserve metadata. This makes your decorators versatile and easier to debug, especially in larger codebases.

4. How do you perform custom aggregation on a pandas groupby() object?

```
import pandas as pd

df = pd.DataFrame({
    'Department': ['IT', 'IT', 'HR', 'HR', 'Finance'],
    'Employee': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'Salary': [80000, 90000, 60000, 65000, 70000],
    'Bonus': [5000, 6000, 3000, 3200, 4000]
})

# Group by Department and perform custom aggregations
agg_df = df.groupby('Department').agg({
    'Salary': ['mean', 'max'],
    'Bonus': lambda x: x.sum() / len(x) # custom aggregation
})
```

Explanation:

- groupby() groups the data based on unique values in the Department column.
- .agg() applies multiple functions per column. Built-ins like 'mean', 'max' and custom ones (like lambda) can be mixed.
- Here, we calculate average salary, max salary, and average bonus manually via a lambda.

Tip:

Use .agg() with dictionaries when you want **column-specific multi-function aggregation**. For advanced needs, wrap lambdas with NamedAgg or use .pipe() for chaining logic cleanly.

5. How does NumPy broadcasting work, and how can it be used to avoid explicit loops?

```
import numpy as np

# Matrix of shape (3, 1)
a = np.array([[1], [2], [3]])

# Vector of shape (1, 4)
b = np.array([10, 20, 30, 40])

# Broadcasting to perform element-wise addition (3x4 matrix)
result = a + b
```

```
print(result)
# Output:
# [[11 21 31 41]
#  [12 22 32 42]
#  [13 23 33 43]]
```

Explanation:

- NumPy **broadcasting** allows operations between arrays of different shapes by automatically **expanding dimensions** where needed.
- Here, a is of shape (3, 1) and b is of shape (4,) → NumPy **broadcasts** them to a common shape (3, 4) for vectorized addition.
- This avoids nested loops and provides a major speed advantage over explicit iteration.

Tip:

Always prefer broadcasting and vectorized operations in NumPy instead of loops — it reduces code complexity and significantly boosts performance, especially in large-scale numerical computations.

6. How do you create a plot with dual y-axes using Matplotlib?

Definition:

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
sales = [100, 120, 130, 150, 170]
profit_margin = [10, 12, 15, 18, 20]

fig, ax1 = plt.subplots()

# First y-axis for sales
ax1.set_xlabel('Year')
ax1.set_ylabel('Sales', color='blue')
ax1.plot(x, sales, color='blue', label='Sales')
ax1.tick_params(axis='y', labelcolor='blue')

# Second y-axis for profit margin
ax2 = ax1.twinx()
ax2.set_ylabel('Profit Margin (%)', color='green')
ax2.plot(x, profit_margin, color='green', label='Profit Margin')
ax2.tick_params(axis='y', labelcolor='green')

plt.title('Sales vs Profit Margin Over Years')
plt.show()
```

Explanation:

- `ax1 = plt.subplots()` creates the base plot.
- `ax1.twinx()` creates a second y-axis sharing the same x-axis.
- Each axis can be customized separately — useful when plotting metrics with **different scales**.
- In this example, we plot sales on the left y-axis and profit margin on the right y-axis.

Tip:

Dual y-axes are helpful when comparing variables with different units/scales. But avoid overuse — too many axes can confuse readers. Always color-code and label both axes clearly.

7. How do you write a Python decorator that accepts arguments?

```
def repeat(n):  
    def decorator(func):  
        def wrapper(*args, **kwargs):  
            for _ in range(n):  
                result = func(*args, **kwargs)  
            return result  
        return wrapper  
    return decorator
```

```
@repeat(3)  
def greet(name):  
    print(f"Hello, {name}!")
```

```
greet("Alice")  
# Output:  
# Hello, Alice!  
# Hello, Alice!  
# Hello, Alice!
```

Explanation:

- To make a decorator accept arguments, you **nest functions three levels deep**:
 1. Outer function: accepts the decorator argument (n)
 2. Middle function: accepts the function being decorated (func)
 3. Inner function: defines the wrapper logic
- @repeat(3) runs the greet() function 3 times using the value passed to the decorator.

Tip:

Always remember: **decorators with arguments require three nested functions**. This pattern is widely used in logging, retry mechanisms, authentication checks, and more.

8. How does NumPy vectorization compare to traditional Python loops in terms of performance?

```
import numpy as np
import time

# Using Python loop
numbers = list(range(1, 1000000))
start = time.time()
squared_loop = [x**2 for x in numbers]
end = time.time()
print("Python loop time:", round(end - start, 4), "seconds")

# Using NumPy vectorization
arr = np.array(numbers)
start = time.time()
squared_np = arr ** 2
end = time.time()
print("NumPy vectorization time:", round(end - start, 4), "seconds")
```

Explanation:

- **Python list comprehensions** iterate one element at a time — they're readable but slower for large computations.
- **NumPy vectorization** leverages optimized C-backed operations and processes entire arrays in bulk.
- In this example, squaring a million numbers is **5–50x faster** with NumPy depending on your machine.

Tip:

For large datasets and numerical operations, always prefer **vectorized NumPy operations** over loops — it's one of the most important optimizations for any data-heavy or ML workload.

9. How do you filter rows in a pandas DataFrame using the .query() method?

```
import pandas as pd
```

```
df = pd.DataFrame({  
    'Department': ['IT', 'HR', 'Finance', 'IT', 'HR'],  
    'Salary': [80000, 60000, 70000, 90000, 65000],  
    'Experience': [5, 2, 3, 7, 1]  
})
```

```
# Filter IT department employees with salary > 85000  
filtered_df = df.query("Department == 'IT' and Salary > 85000")
```

Explanation:

- .query() lets you filter rows using a string-based expression, similar to SQL WHERE clauses.
- It avoids long boolean indexing chains like `df[(df['Department'] == 'IT') & (df['Salary'] > 85000)]`.
- Internally, it parses the query string and evaluates it efficiently using pandas' eval engine.

Tip:

Use .query() for cleaner, more readable code, especially when filtering using multiple conditions. Just remember: column names with spaces must be enclosed in backticks (` `).

10. What is the difference between Seaborn and Matplotlib, and when should you use each?

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

# Sample data
df = pd.DataFrame({
    'Department': ['IT', 'HR', 'Finance', 'IT', 'HR'],
    'Salary': [80000, 60000, 70000, 90000, 65000],
    'Experience': [5, 2, 3, 7, 1]
})

# Seaborn: High-level API
sns.barplot(x='Department', y='Salary', data=df)
plt.title('Average Salary by Department')
plt.show()

# Matplotlib: Lower-level control
departments = df['Department'].unique()
avg_salary = df.groupby('Department')['Salary'].mean()
plt.bar(departments, avg_salary)
plt.title('Average Salary by Department (Matplotlib)')
plt.xlabel('Department')
plt.ylabel('Average Salary')
plt.show()
```

Explanation:

- **Matplotlib** is the base library for plotting in Python. It offers full control but requires more code for formatting.
- **Seaborn** is built on top of Matplotlib and provides high-level functions with beautiful default themes, especially for **statistical plots**.
- In the examples above, Seaborn handles grouping and aesthetics automatically, while Matplotlib requires manual steps.

Tip:

Use **Seaborn** for quick, publication-ready plots and **Matplotlib** when you need fine-grained control over axes, ticks, annotations, or figure composition. In practice, they are often used **together**.

11. How do you use MultiIndex in pandas and reshape data using stack() and unstack()?

Definition:

```
import pandas as pd
```

```
# Sample multi-level data
df = pd.DataFrame({
    'Department': ['IT', 'IT', 'HR', 'HR'],
    'Year': [2022, 2023, 2022, 2023],
    'Salary': [80000, 85000, 60000, 62000],
    'Bonus': [5000, 5500, 3000, 3200]
})

# Set MultiIndex
df_multi = df.set_index(['Department', 'Year'])

# Reshape: unstack year → columns become 2022 and 2023
reshaped = df_multi['Salary'].unstack()

print(reshaped)
```

Explanation:

- `set_index()` creates a **MultiIndex** from Department and Year, giving you hierarchical row labels.
- `unstack()` pivots one level of the index (here: Year) into columns.
- You can use `stack()` to reverse this operation — flattening columns back into a deeper index.

Tip:

Use **MultiIndex** and `stack()/unstack()` when working with time series, pivot tables, or grouped data across multiple dimensions — they provide powerful reshaping without loops or manual joins.

12. How do you define and use custom exceptions in Python, and integrate them with logging?

```
import logging

# Configure logging
logging.basicConfig(level=logging.INFO, format='%(levelname)s - %(message)s')

# Define a custom exception
class InvalidSalaryError(Exception):
    def __init__(self, salary):
        self.salary = salary
        super().__init__(f'Invalid salary: {salary}. Salary must be positive.')

# Function that uses the custom exception
def process_salary(salary):
    if salary <= 0:
        logging.error("Attempted to process an invalid salary.")
        raise InvalidSalaryError(salary)
    logging.info(f"Processing salary: {salary}")
    return salary * 1.1 # hypothetical salary hike

try:
    process_salary(-50000)
except InvalidSalaryError as e:
    logging.exception("Handled exception:")
```

Explanation:

- Custom exceptions allow you to define meaningful, **domain-specific errors**.
- InvalidSalaryError extends Exception, with a custom message and attribute.
- logging provides visibility into the system's behavior and errors — far better than print() in real-world applications.

Tip:

Use **custom exceptions** to clarify error intent and **logging** to trace and debug errors without breaking execution flow — especially important in APIs, ETL jobs, or multi-layer systems.

13. What is the difference between `is` and `==` in Python?

```
a = [1, 2, 3]
b = a
c = [1, 2, 3]

print(a == c) # True – values are equal
print(a is c) # False – different objects
print(a is b) # True – same object reference
```

Explanation:

- `==` checks **value equality** — whether the contents are the same.
- `is` checks **identity** — whether two variables point to the same object in memory.

Tip:

Use `==` to compare values, and `is` to check identity (e.g., if `x` is `None`). Overusing `is` for equality checks is a common bug in beginner code.

14. How do you check if a key exists in a dictionary?

```
my_dict = {'name': 'Alice', 'age': 25}
```

```
# Check key existence
if 'name' in my_dict:
    print("Key exists!")
```

Explanation:

- The `in` keyword checks if a key exists in a dictionary.
- This is a safe way to access keys and avoid `KeyError`.

Tip:

Avoid using `try...except KeyError` for control flow unless absolutely necessary — use `'key' in dict` instead.

15. What is list comprehension in Python?

```
# Generate a list of squares from 0 to 4
squares = [x**2 for x in range(5)]
print(squares) # Output: [0, 1, 4, 9, 16]
```

Explanation:

- List comprehension is a compact way to build lists using a single line of code.
- It's equivalent to writing a `for` loop and appending to a list.

Tip:

Use list comprehensions for cleaner, faster code — and add conditions like `[x for x in range(10) if x % 2 == 0]` for filtered results.

16. How do you remove duplicates from a list?

```
my_list = [1, 2, 2, 3, 4, 4, 5]
unique = list(set(my_list))
print(unique) # Output: [1, 2, 3, 4, 5] (order not guaranteed)
Explanation:
```

Converting a list to a set removes duplicates since sets can't have repeating values.

Use `list()` to convert back to list.

Tip:

To preserve order, use:

```
list(dict.fromkeys(my_list))
```

17. How do you use a pandas `apply()` function with a custom lambda or function?

```
import pandas as pd
```

```
df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Score': [85, 90, 95]
})
```

```
# Using apply with lambda to add grade
df['Grade'] = df['Score'].apply(lambda x: 'A' if x >= 90 else 'B')
```

```
print(df)
```

Explanation:

- The `apply()` function applies a **custom function row-wise** or column-wise.
- Here, we use a **lambda** to assign a grade based on Score.
- It's more flexible than `map()` when logic is conditional or depends on multiple columns.

Tip:

When applying functions across rows with multiple columns involved, use `axis=1`:
`df.apply(lambda row: row['Score'] + len(row['Name']), axis=1)`

18. How do you use `groupby().transform()` to perform group-level operations while keeping original row structure?

```
import pandas as pd

df = pd.DataFrame({
    'Team': ['A', 'A', 'B', 'B'],
    'Player': ['P1', 'P2', 'P3', 'P4'],
    'Score': [10, 20, 30, 40]
})

# Normalize score within each team
df['Normalized'] = df['Score'] / df.groupby('Team')['Score'].transform('sum')

print(df)
```

Explanation:

- `transform()` returns a **Series aligned to the original DataFrame**, allowing row-wise operations.
- Here, we normalize each player's score by the **team total**.
- Unlike `agg()`, `transform()` retains the same number of rows.

Tip:

Use `transform()` for **feature engineering**, like z-score, ratio, or percent contribution within a group — while keeping the DataFrame intact.

19. How do you use .pipe() in pandas to write clean, modular transformation chains?

Assume a transactions table:

```
import pandas as pd
```

```
df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Score': [82, 91, 78]
})
```

```
# Custom transformation function
```

```
def add_pass_fail(df):
    df['Status'] = df['Score'].apply(lambda x: 'Pass' if x >= 80 else 'Fail')
    return df
```

```
# Another transformation
```

```
def boost_scores(df, increment):
    df['Score'] += increment
    return df
```

```
# Using .pipe() for clean chaining
```

```
result = (
    df
    .pipe(boost_scores, increment=5)
    .pipe(add_pass_fail)
)
```

```
print(result)
```

Explanation:

- .pipe() allows you to **chain custom functions** in a clear, functional style — each function receives the DataFrame as input.
- Unlike deeply nested function calls, pipe() maintains readability.
- Parameters like increment=5 are passed after the DataFrame.

Tip:

Use .pipe() to build **modular, testable ETL or preprocessing steps** — especially useful when functions are reused across pipelines or notebooks.

Data Analysis/Scenario-Based Questions

20. Scenario: You receive a list of user login records. Some users logged in multiple times on the same day. Write a Python script to find users who logged in more than once per day.

```
from collections import defaultdict

logins = [
    ("alice", "2024-05-10"),
    ("bob", "2024-05-10"),
    ("alice", "2024-05-10"),
    ("charlie", "2024-05-11"),
    ("alice", "2024-05-11"),
    ("bob", "2024-05-11"),
    ("bob", "2024-05-11")
]

# Count logins per (user, date)
login_count = defaultdict(int)

for user, date in logins:
    login_count[(user, date)] += 1

# Filter users with multiple logins on the same day
duplicates = {key for key, count in login_count.items() if count > 1}

print("Users with multiple logins on the same day:")
for user, date in duplicates:
    print(f"{user} on {date}")
```

Explanation:

We use a `defaultdict(int)` to count (user, date) pairs.

This simulates a grouping operation — similar to `groupby` in pandas but done in pure Python.

Finally, we filter keys where the count exceeds 1.

Tip:

Scenario questions like this test your ability to simulate SQL or pandas logic using core Python.

Prioritize clarity: use collections like `defaultdict` or `Counter` for clean logic instead of nested loops.

21. Scenario: You are given a system log file. Each line contains a timestamp and a log level. Your task is to parse the file and extract all lines where the log level is "ERROR"

Sample logs.txt content:

```
2024-07-01 10:02:15 [INFO] System started
2024-07-01 10:05:22 [ERROR] Failed to load config
2024-07-01 10:07:30 [WARNING] Low memory
2024-07-01 10:10:45 [ERROR] Connection timeout
```

```
def extract_error_logs(file_path):
    with open(file_path, 'r') as f:
        lines = f.readlines()

    error_logs = [line.strip() for line in lines if '[ERROR]' in line]
    return error_logs
```

```
# Simulate reading from file
sample_logs = """
2024-07-01 10:02:15 [INFO] System started
2024-07-01 10:05:22 [ERROR] Failed to load config
2024-07-01 10:07:30 [WARNING] Low memory
2024-07-01 10:10:45 [ERROR] Connection timeout
"""
```

```
# Writing the logs to a file (in real case, logs.txt would already exist)
with open('logs.txt', 'w') as f:
    f.write(sample_logs.strip())
```

```
# Extract error logs
errors = extract_error_logs('logs.txt')
for line in errors:
    print(line)
```

Explanation:

- This script reads the file line-by-line and filters entries that contain "[ERROR]".
- `.strip()` removes newline characters or extra spaces.
- This is a typical pre-processing step before sending alerts, logging metrics, or dashboarding.

22. Scenario: You are given a list of user records. Some names have extra spaces, inconsistent casing, or are missing. Write a script to clean the data.

Assume a transactions table:

```
raw_users = [  
    {'name': ' Alice ', 'email': 'alice@example.com'},  
    {'name': 'bob', 'email': 'bob@example.com'},  
    {'name': None, 'email': 'charlie@example.com'},  
    {'name': ' CHARLIE', 'email': 'charlie2@example.com'},  
    {'name': 'david', 'email': 'DAVID@example.com'},  
]
```

```
def clean_user_data(users):  
    cleaned = []  
    for user in users:  
        name = user['name']  
        if name is None:  
            continue # skip records with missing names  
  
        clean_name = name.strip().title()  
        cleaned.append({  
            'name': clean_name,  
            'email': user['email'].lower()  
        })  
    return cleaned
```

```
cleaned_users = clean_user_data(raw_users)  
for user in cleaned_users:  
    print(user)
```

Explanation:

- `.strip()` removes unwanted whitespace from names.
- `.title()` standardizes name casing (e.g., " CHARLIE" → "Charlie").
- `.lower()` ensures email addresses are case-insensitive.
- None values are filtered out early to avoid downstream errors.

Tip:

In interviews, emphasize **defensive programming** — always check for None, invalid types, or unexpected formats when cleaning raw data. You'll stand out if you mention **reusability**, like wrapping it in a function or pipeline step.